

Day 7 — RTL Specification Materials

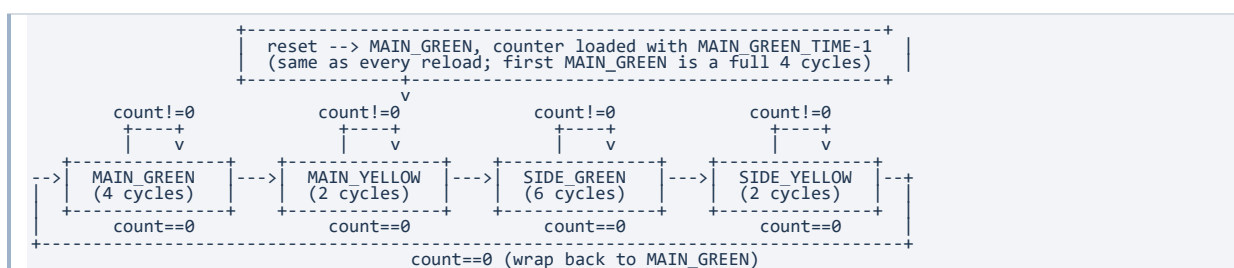
Sequence detector interface, traffic-light state diagram, corner cases for all three RTLs, and the FIFO overview.

Item 3 — Interface Table: RTL #1 (Sequence Detector)

Every port, every column filled, per Learning Topic 7.3's requirement that a description state *when* a signal is asserted and *what it causes* — not just restate the port name.

Port	Direction	Width	Clock Domain	Active Level	Description
<code>clk</code>	input	1	— (source)	rising-edge	System clock. All sequential logic in this design (state register, output register) updates on the rising edge of <code>clk</code> .
<code>rst_n</code>	input	1	<code>clk</code>	active-low	Asynchronous reset. When LOW, forces the FSM to <code>IDLE</code> and <code>detected</code> to 0, independent of <code>clk</code> activity.
<code>din</code>	input	1	<code>clk</code>	active-high, sampled	Serial data input bit, sampled once per rising edge of <code>clk</code> . The FSM tracks how much of the target sequence <code>1011</code> has been matched using this bit.
<code>detected</code>	output	1	<code>clk</code>	active-high, one-cycle pulse	Pulses HIGH for exactly one clock cycle when the FSM confirms <code>din</code> has completed an occurrence of <code>1011</code> (overlapping detection — see Day 2/4). It is a Moore output : a pure combinational decode of the current state (<code>detected = (state == S1011)</code>), HIGH only while the FSM sits in the dedicated DETECT state <code>S1011</code> , with no dependency on <code>din</code> . Because <code>S1011</code> is entered the cycle after the 4th matching bit is sampled, the pulse is one clock of ordinary FSM latency after that bit — confirmed via simulation in experiments/seq_detect_1011_smoke_log.md (pulse in cycle 8, in state <code>S1011</code>).

Item 4 — ASCII State Diagram: Traffic Light Controller



Notes

- Every self-loop (`count!=0`) means “stay in this state, the down-counter just decremented.” Every forward arrow (`count==0`) means “this phase’s duration just expired, advance to the next phase in the fixed sequence.”
- There is no external input anywhere in this diagram — the *only* thing that ever drives a transition is the counter reaching zero (Learning Topic 5.1’s FSM+datapath pattern).
- Reset forces `MAIN_GREEN` and loads the counter with `MAIN_GREEN_TIME - 1` — **exactly the same value a normal reload uses** — so the first `MAIN_GREEN` phase lasts a full `MAIN_GREEN_TIME` (4) cycles, identical to every later `MAIN_GREEN`. The first draft loaded `MAIN_GREEN_TIME` (not -1) at

reset, which made the first phase 5 cycles; see [ISSUE-D5-02](#) (corrected). A duration of D cycles corresponds to the counter stepping D-1, D-2, ..., 0, so the reload value is always D-1, reset included.

Item 2 — Corner Cases: All Three RTLs

At least 3 per design, per the assignment (I've listed 4 each for margin).

RTL #1 — Sequence Detector

#	Corner Case	Expected Behavior
1	Overlapping match immediately after a previous match (e.g., ...1011011...)	Both occurrences of 1011 must be detected. Confirmed by the dedicated self-checking overlap regression (seq_detect_1011_overlap_tb) driving 1011011 — two detections fire, in cycles 5 and 8 (3-cycle gap, the minimum for overlapping 1011). Note: the Day-4 smoke vector 1101011 does not exercise this — it contains only one 1011 — which is why a dedicated overlap test exists.
2	Reset asserted mid-sequence (e.g., FSM is in S101 when rst_n drops)	FSM must return cleanly to IDLE, detected must go LOW, and any partial match progress must be discarded — not carried across reset.
3	din held constant (all 0s or all 1s) for many cycles	Must never falsely assert detected; must correctly self-loop (IDLE on all-0s, S1 on all-1s) indefinitely without erroring.
4	First bit after reset is a 1	FSM must immediately begin progressing from IDLE toward S1 — no “warm-up” cycle required before the FSM starts tracking progress.

RTL #2 — Traffic Light Controller

#	Corner Case	Expected Behavior
1	Reset asserted while mid-countdown in a non-initial state (e.g., reset hits during SIDE_GREEN)	Must always return to MAIN_GREEN and reload the counter with MAIN_GREEN_TIME - 1 (the same value a normal reload uses), giving a full 4-cycle MAIN_GREEN regardless of which state or count value reset happened to catch the FSM at.
2	Wraparound from SIDE_YELLOW back to MAIN_GREEN	Must reload for a full MAIN_GREEN_TIME duration every time this state is re-entered — not just the first time. The 20-cycle smoke test (one full 14-cycle loop plus 6 cycles into the second) now shows the first and second MAIN_GREEN both lasting a true 4 cycles, verifiable from the exposed count column. Note: it was the corrected logging (honest per-cycle sampling + count column, ISSUE-D5-03) that made ISSUE-D5-02 visible — the earlier negedge-loop sampling had masked it by dropping the first cycle from the printout.
3	A duration parameter set to the minimum realistic value (1 cycle)	The load-vs-decrement priority (count == 0 check) must still advance the FSM exactly one cycle later, without skipping the state or holding an extra cycle. Confirmed by the min-duration regression test (traffic_light_min_duration_tb, MAIN_YELLOW_TIME=1), which shows MAIN_YELLOW occupying exactly one cycle (count loaded to 0, reloads next edge) between a 4-cycle green and a 6-cycle green — closing ISSUE-D7-01 .
4	Two adjacent states sharing the same duration value	The counter reaching zero must still trigger a transition to the correct next state (per the fixed sequence), not “confuse” which state comes next just because two durations happen to coincide numerically.

RTL #3 — Synchronous FIFO

#	Corner Case	Expected Behavior
1	Write while full	Blocked — no data corruption, count never exceeds DEPTH. Confirmed in Day 6 smoke test (poison value 0xFF never appeared in any later read).
2	Read while empty	Blocked — no count underflow. Confirmed in Day 6 smoke test.

#	Corner Case	Expected Behavior
3	Simultaneous read+write, in each of the full / empty / partially-filled sub-cases	<code>count</code> behaves correctly in all three: unchanged when partially filled, decrements-only when full, increments-only when empty. All three confirmed via the extended Day 6 testbench (ISSUE-D6-01).
4	Pointer wraparound exactly at the <code>DEPTH</code> boundary	Writing/reading across the “seam” where <code>wr_ptr</code> or <code>rd_ptr</code> wraps from <code>DEPTH-1</code> back to <code>0</code> must not skip or duplicate a memory slot. Confirmed by the self-checking multi-wrap regression (sync_fifo_multiwrap_tb , ISSUE-D7-02): 3 consecutive fill-then-drain cycles (24 writes / 24 reads, both pointers wrapping multiple times) with every read matched against a software reference queue — PASS .

Item 5 — One-Paragraph FIFO Overview

For a DV engineer who has never seen it.

This is a standard single-clock-domain synchronous FIFO with 8 entries and 8 bits of width (both parameterizable), featuring write-side control (`wr_en`, `din`), read-side control (`rd_en`, `dout`), and `full` and `empty` status flags; internally it comprises an 8-entry memory array, write and read pointers, and a count register that directly controls the status flags since comparing pointers alone cannot distinguish between the completely-empty and completely-full states (both make the pointers equal). Writes are blocked when the FIFO is full and reads when empty, but the design correctly handles simultaneous `wr_en` and `rd_en` assertions in the same cycle — both proceed without changing the entry count, except in edge cases where the FIFO is already completely full (only the read executes, freeing a slot) or completely empty (only the write executes). Note that `dout` is a combinational “peek” at the read pointer’s target rather than a registered output, updating immediately; when `empty=1`, it displays stale leftover data rather than meaningful content, so any test reading `dout` must gate the check on `empty` first.